

Exam 2

summary
sheet 0

Boolean

- 1/0
- True / False values
- Constants in python

Boolean Expressions

We can ask questions using functions that return true/false answers, or with a set of boolean comparison operators. These form boolean expressions that evaluate to True or False. The most common boolean operators are shown in the table below.

$A == B$	"Is equal to" takes two expressions, A and B, and evaluates to True if the value of A is equal to the value of B
$A != B$	"Is not equal to" takes two expressions, A and B, and evaluates to True if the value of A is not equal to the value of B
$A <= B$	"Is less than or equal to" takes two expressions, A and B, and evaluates to True if the value of A is less than or equal to the value of B
$A < B$	"Is less than" takes two expressions, A and B, and evaluates to True if the value of A is strictly less than the value of B
$A >= B$	"Is greater than or equal to" takes two expressions, A and B, and evaluates to True if the value of A is greater than or equal to the value of B
$A > B$	"Is greater than" takes two expressions, A and B, and evaluates to True if the value of A is greater than the value of B
$A \text{ in } B$	"Is in" or "Is an element of" takes two expressions, A and B, and evaluates to True if the value of A occurs in the value of B For strings this asks if A is a substring of B For lists, this asks if A is an element of B

Conditional Statements

if boolean-expression:
code-to-perform
elif boolean-expression:
code-to-perform

no boolean expression with else
else:
code-to-perform

when the if statement is true the else is omitted

Precedence of Operators

Level	Category	Operator
7 (high)	exponent	$*$
6	mul/div	$*, /, //, \%$
5	add/sub	$+, -$
4	boolean expression	$==, <, <=, >, >=$
3	logical operator	not
2	"	and
1 (low)	"	or

Nested Conditionals

This is when another if statement is put within an if statement as a condition to a boolean response i.e. whether or not the previous statement is True or False

elif is a type of a nested conditional statement called chained conditionals
→ it's an abbreviation of else B if = elif

Boolean Logical Operators

We can also put boolean expressions like the ones above together using the boolean logical operators: and, or, and not. The table below describes how these operators work.

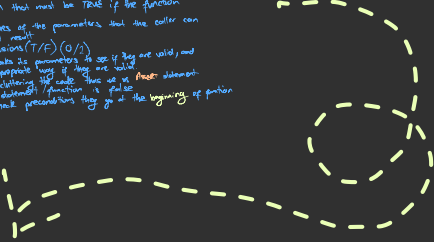
$A \text{ and } B$	Evaluates to True if both the value of A and the value of B are True, otherwise it evaluates to False
$A \text{ or } B$	Evaluates to True if at least one of A or B evaluates to True, otherwise it evaluates to False
not A	Evaluates to True if the value of A is False, and to False if the value of A is True

Unit Test

This is more about methodical testing

Assert

A **precondition** specifies a condition that must be true if the function is to produce correct results.
 It is a constraint on the values of the parameters that the caller can pass. It expects to produce a valid result.
 A **postcondition** is a condition that must be true if the function is to produce correct results.
 It is a constraint on the values of the parameters that the caller can pass. It expects to produce a valid result.
 A **precondition** is a function that checks its parameters to see if they are valid, and if not, it raises an exception.
 A **postcondition** is a function that checks the results of the function to see if they are correct.
 A **test** is a small program that checks if the function produces the expected results.
 A **test** will give an error if the function produces a failure.
 A **test** will give an error if the function produces a failure.
 A **test** will give an error if the function produces a failure.



Manual Unit Test → gets input from user
 → you have to look at result of output to determine whether or not the test passed / failed
Automated Unit Test → contains the part that input
 → produces a clear pass / fail indication

EXAMPLE

A)

```
def round6(num):
    return int(num + 0.5)
result = round6(9.7)
if result == 10:
    print("Test 1 PASS")
else:
    print("Test 1 FAIL")
```

B)

```
def round6(num: float) -> int:
    return int(num + 0.5)
result = round6(9.7)
assert result == 10
```

both produce the same result but B is more efficient

PS: Sometimes it's the unit test with an error and not the function

Testable Function

A function that produces a result that can be checked by an assert statement. It does one of the following:

- it returns its result
- it stores its result in a global variable
- it modifies the state of an object passed as a parameter

PS: functions that display their output using print are **NOT** testable functions

A **postcondition** states the work the function completed by the function if the precondition is satisfied.

Writing Unit Tests

Specification-Based Tests

→ they are tests that are designed based only on the info in the function specification, without considering any of the details in the function implementation.

★ They are prepared over implementation-based tests, because they are more resilient. They will continue to pass even if you change the function implementation.

```
def sum_students(s):
    """Returns the sum of a tuple of students"""
    # Assert that s is a tuple
    assert isinstance(s, tuple), "The sum of the students is the sum of the students"
    # Assert that s contains only integers
    for i in s:
        assert isinstance(i, int), "The sum of the students is the sum of the students"
    # Return the sum of the students
    return sum(s)

# We've seen, to write a unit test, you decide test cases for the function, and then write assert
# statements that call the function and check that the function produced the expected results. The
# following assert statements would be appropriate for a unit test for sum_students:

result = sum_students(1, 2) == 3
assert sum_students(1, 2) == 3
```

20.6.1. Benefits of Test-First Development

There are several benefits to Test-First Development.

- It ensures that unit tests are written. This tends to lead to higher-quality, robust code, with fewer bugs.
- Writing the tests first helps the programmer to clarify the function specification. It's not possible to write an assert for a function that has a vague function description. This process forces the programmer to write a clear description and to practice specification-based testing. Because when the tests are written, there is no function implementation to reference.
- When the programmer writes the function and is ready to test it, the test is all ready to go. There is no internal struggle about whether a unit test should be written or not. The programmer runs the test, and gets instant feedback about whether the function is working or not.
- If the function fails to pass the test, the benefits of unit testing in helping the programmer to quickly diagnose and fix the problem are instantly available. The test-debug cycle is rapid.
- When a programmer modifies an existing function for which unit tests already exist, perhaps to add some more functionality, the existing unit tests serve as a safety net. They check that the modifications made by the programmer don't break any of the old functionality.
- The overall development time tends to be reduced. Perhaps counter-intuitively, writing more code (the unit tests) actually speeds up the overall development process, because of the benefits imparted by unit testing.
- Believe it or not, there are psychological benefits. As the programmer works on the project, creating little tests and then writing code that passes those tests, there is a sense of accomplishment and satisfaction that comes every time a new test passes. Instead of spending hours of frustration debugging a new function in the context of a complex program, with few visible results, the test-first progress leads to more visible and regular successes.

Pytest

→ simple assert statements don't give you good diagnostic information when they fail.
 → we need a better way to create unit tests that give us good diagnostic information when they fail.
 → we need a way to keep our test code separate from the function code that we are testing.
 → we need a way to keep our test code separate from the function code that we are testing.

Enter **Pytest**

How?

Basically the same thing except that you put your assert statements and test functions

```
1 def round6(num: float) -> int:
2     """This function has a bug in it"""
3     return int(num + .5)
4
5 # --- automated unit test ---
6
7 def test_round6():
8     assert round6(9.7) == 10
9     assert round6(8.5) == 8
10
11
```

You can do this in a different file and call your functions from a different file

[cool right?]

functions that include a precondition and a postcondition their description embody software engineering idea called **DESIGN**

33 Context!!!

Iteration

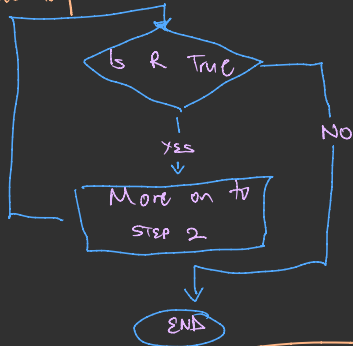
This is a repeated execution of a sequence of statements.

- A for loop processes each item in a list eg

```
for f in ["Den", "Prov"]:  
    print("Hi", f)
```

The result would be
Hi Den
Hi Prov

- While loop



Ps:

With a while loop the value of 1 or more variables has to change so that eventually the condition becomes FALSE, otherwise the loop with repeat indefinitely.

Infinite Loop

Conclusion

for loop is better than while
Why: cause in for loop you don't have to control the variable to prevent an infinite loop like you have to do in a while loop.



If you wanted to create a turtle that wanders around randomly inside the screen you'd use a while loop.

```
create a window and a turtle  
  
while the turtle is still in the window:  
    generate a random number between 0 and 1  
    if the number == 0 (heads):  
        turn left  
    else:  
        turn right  
    move the turtle forward 50
```

Newton's Method
→ its a variation of an accumulator pattern

```
1 def newtonSqrt(n, howmany):  
2     approx = 0.5 * n  
3     for i in range(howmany):  
4         betterapprox = 0.5 * (approx + n/approx)  
5         approx = betterapprox  
6     return betterapprox  
7  
8 print(newtonSqrt(100, 10))  
9 print(newtonSqrt(4, 10))  
10 print(newtonSqrt(1, 10))
```

Use a while loop to validate input eg:

```
1 def get_yes_or_no(message):  
2     valid_input = False  
3     answer = input(message)  
4     while not valid_input:  
5         answer = answer.upper() # convert to upper case  
6         if answer == 'Y' or answer == 'N':  
7             valid_input = True  
8         else:  
9             answer = input('Please enter Y for yes or N for no. \n' + message)  
10    return answer  
11  
12 response = get_yes_or_no('Do you like lima beans? Y (yes) or N (no): ')  
13 if response == 'Y':  
14     print('Great! They are very healthy.')15 else:  
16     print('Too bad. If cooked right, they are quite tasty.')17
```

String Methods

• Strings are immutable, you cannot modify an existing string, but you can construct a modified copy of the string.
Some string methods that can be used include:

- ① string methods
- ② concatenation
- ③ star repetition
- ④ loop

• strings are made of smaller pieces called character
↳ strings with no characters are called empty string
↳ represented by "" or ''
↳ can perform mathematical operations on strings even if the strings look like numbers.
↳ you cannot combine the strings and add them up
↳ `opening + 1` repeats the string

eg. `school[2]`
string `o` `s` `h` `o` `o` `l` `h` `o` `o` `l`
`s = school`
`print(s[2])` → position number of requested character in position 2
printed `s` will be `h` → expression in brackets is called an index

Can find the list of string tools in the documentation

'\t' is a tab character

↳ indicates the beginning of an escape sequence

'\n' → represents new line

Capitalization-related

The methods here are used both to check on the capitalization of a string, and to change its capitalization. None of these take any inputs, but they do return values.

Method	Description
<code>islower</code>	Returns True if all characters in its string are lowercase and False otherwise
<code>isupper</code>	Returns True if all characters in its string are uppercase, and False otherwise
<code>istitle</code>	Returns True if the string is a title with every first character capitalized, and no others, and it returns False otherwise
<code>capitalize</code>	Returns a copy of the string with its first character capitalized and the rest lowercase
<code>lower</code>	Returns a copy of the string with all characters lowercase
<code>upper</code>	Returns a copy of the string with all characters uppercase
<code>title</code>	Returns a copy of the string with the first character of each word capitalized and all the rests lowercase
<code>swapcase</code>	Returns a copy of the string with the case of each letter swapped: lower to upper and upper to lower

Search/Replace-related

These methods look for certain substrings or characters in a string, and sometimes replace them with new strings. Remember that they always generate a new string to do a replacement, they do not ever change the input string, because strings are immutable.

Method	Description
<code>find</code>	Takes in a string, and returns the index of the leftmost occurrence of that string inside its string or -1 if it isn't there
<code>rfind</code>	Similar to <code>find</code> , but returns the index of the rightmost occurrence
<code>endswith</code>	Basic version takes in a string and return True if its string ends with the input string. For options, see documentation.
<code>startswith</code>	Basic version takes in a string and return True if its string starts with the input string. For options, see documentation.
<code>index</code>	Another name for <code>find</code> for compatibility with list methods
<code>rindex</code>	Another name for <code>rfind</code> for compatibility with list methods
<code>count</code>	Takes in a string, and returns the number of occurrences of that string in its string (no overlaps)
<code>replace</code>	Takes in two strings, and it builds a new string where every occurrence in its string of the first input has been replaced by the second input

Spacing-related

These methods are used to add or remove spaces (or other characters) from the front or end of the string to change the spacing around the main contents of the string.

Method	Description
<code>center</code>	Takes in a new width and an optional fill character, and it returns a new string where the original string is centered in the new one. By default, extra characters are filled with spaces, or with the fill character if given. If width is shorter than the string, than the original string is returned.
<code>ljust</code>	Similar to <code>center</code> , but it adds spaces/fill chars to the right end
<code>rjust</code>	Similar to <code>center</code> , but it adds spaces/fill chars to the left end
<code>strip</code>	Takes in an optional string. If given no input, it removes whitespace from front and end of the string. If input string is provided, then it removes those characters, if they occur, from front and end of the string.
<code>lstrip</code>	Similar to <code>strip</code> , but only removes from front of string
<code>rstrip</code>	Similar to <code>strip</code> , but only removes from end of string

Content Check-related

The following set of methods ask about the contents of the string, and are useful for determining whether a string is in a useful format or not. None of these methods take in any inputs.

Method	Description
<code>isalpha</code>	Returns True if all the characters in its string are alphabetic (all letters)
<code>isalnum</code>	Returns True if all the characters in its string are either alphabetic or numerical
<code>isdigit</code>	Returns True if all the characters in its string are digits
<code>isspace</code>	Returns True if all the characters in its string are "whitespace" space, tab, or newline

Breaking into Pieces

Often when we have a piece of text, we want to break it up into words or lines. Or if we have other structured data like CSV files, we might want to split it by commas or tabs. These methods let you separate a string into parts.

Method	Description
<code>split</code>	Takes an optional input, a character. Returns a list of strings, created by splitting up its string. By default it splits on whitespace, but with an input it splits on that character.
<code>splitlines</code>	Similar to <code>split</code> but it only splits on newline, returning a list of strings for each line

Lists

A Comprehensive Guide to Python Lists



1. List Fundamentals



Creating a List

A collection data type that holds an ordered sequence of other values. It is one of Python's most common native datatypes.

Example:

```
[10, "hello", 3.14, True]
```

Accessing Elements by Index

`my_list[0]` returns the first item.

Determining List Length

The built-in `len()` function returns the total number of items.

```
len(my_list)
```

Checking for Membership

The `'in'` and `'not in'` operators return `'True'` or `'False'` if an item exists within a list.

```
"hello" in my_list
```



2. Core List Operations



Concatenation (+)

The `'+'` operator joins two lists together to create a new, combined list.

```
[1, 2] + [3, 4]
```

Repetition (*)

The `'*'` operator creates a new list by repeating the original list's elements.

```
[0] * 4
```

Slicing (:)

The slice operator `[':']` extracts a subsection, creating a new list.

```
my_list[1:3]
```

Deletion (del)

The `'del'` statement removes an element at a specific index, modifying the list in place.

```
del my_list[2]
```



3. Mutability: The Most Important Concept

Lists are Mutable

Unlike strings, lists can be changed after they are created. You can change, add, or remove elements.

Objects and References

Variable name pointing not to the list directly.

Memory location

```
my_list
```

Aliasing: Multiple References, One Object

When you assign one list variable to another (`b = a`), you are creating an alias. Both variables point to the same list object. Changing one will affect the other.

Cloning: Creating a Separate Copy

To avoid aliasing, you must clone the list. The easiest way is to use the slice operator with no indices:

```
new_list = original_list[:]
```



4. Working with Lists: Methods & Loops



Common List Methods

Methods are functions attached to an object. Most list methods modify the list in place and return `None`.

Method	Description	Example
<code>append(item)</code>	Adds a single item to the end of the list.	<code>my_list.append(0)</code>
<code>insert(i, item)</code>	Inserts an item at a specific index.	<code>my_list.insert(0, 'start')</code>
<code>pop()</code>	Removes and returns the item at the end of the list. If <code>i</code> is specified, removes the item at <code>i</code> .	<code>last_item = my_list.pop()</code>
<code>sort()</code>	Sorts the list in place (ascending by default).	<code>my_list.sort()</code>
<code>reverse()</code>	Reverses the order of elements in the list in place.	<code>my_list.reverse()</code>
<code>index(item)</code>	Returns the index of the first occurrence of an item.	<code>pos = my_list.index('hello')</code>
<code>count(item)</code>	Returns the number of times an item appears in the list.	<code>num = my_list.count(3)</code>

Iterating with 'for' Loops

```
for item in my_list:  
    print(item)
```

The Accumulator Pattern

```
total = 0  
for num in numbers:  
    total += num
```

A common pattern where you initialize an "accumulator" variable and update it during each iteration to produce a final result.



5. Advanced Topics & Related Structures



List Comprehensions

A concise and powerful syntax for creating a new list by applying an expression to each item in another.

```
squares = [x**2 for x in range(10)]
```

A consists and powerful for expression for to each item in another sequence.

Nested Lists

A list can contain other lists as elements, which is useful for representing grids or matrices (2D data).

Converting Between Strings and Lists

```
split() -> ['s', 'l', 't', 'w', 'o', 'r', 'd', 's']  
list() -> ['s', 'l', 't', 'c']
```

Tuples: The Immutable Cousin

Immutable: cannot be changed after creation

Tuples are like lists but are immutable. Use them when data should not be modified.

Tuple Assignment

```
(name, age) = ("Alice", 30)
```

A convenient feature that allows you to assign values from a tuple (or list) to multiple variables at once.

Dictionaries

A Programmer's Guide to Dictionaries: Concepts from Key Python Texts

Locating "Dictionary" Concepts in Programming Texts



Chapter 12 Dictionaries

Explicitly titled dedicated chapter. Provides detailed breakdown of operations, methods, and applications.

Dive Into Python 3

Includes dictionaries under broader topic of Native Datatypes as a core feature.



Chapter 12 Collections

Discusses related abstract concepts and principles behind dictionary-like data structures.

Deep Dive: An Outline of a Dictionary Chapter



The Core Concept: What is a Dictionary?

Introduce fundamental nature of dictionaries as key-value pair collections.

Mastering Dictionary Operations

Covers basic actions for manipulating dictionaries: adding, accessing, or removing pairs.

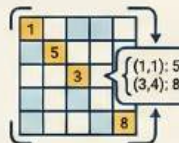


Using Built-in Dictionary Methods

Learn about powerful built-in functions like .keys(), .values(), and .items() to work with dictionary data efficiently.

Understanding Aliasing vs. Copying

Explores critical difference between creating a reference and an independent copy.



Practical Application: Sparse Matrices

Common use case demonstrating efficient representation of mostly empty data structures.

Solidifying Knowledge: Glossary & Exercises

Concludes with key terminology and practice problems to test and reinforce understanding.



Files

Python File I/O: A Chapter Summary



GETTING STARTED WITH FILES



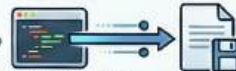
Working with Data Files
Introduces the fundamental concepts of using data files as a source of input for programs.



Finding a File on your Disk
Covers the necessary steps and considerations for locating a file within your computer's file system for your program to access.



WRITING DATA TO FILES



Writing Test Files
Describes how to open files in write or append mode to save program output or create new text files.

READING DATA FROM FILES



Reading a File
Explains the primary methods and functions used to open a file and read its contents into your program.



Iterating Over Lines in a File
Details the common and efficient pattern of processing a file one line at a time using a loop.



Alternative File Reading Methods
Presents other available methods for reading files, likely including reading the entire file at once or reading a specific number of bytes.



BEST PRACTICES & RELATED TOPICS



Using "With" Statements
Emphasizes the use of the 'with' statement for safely handling files, which ensures they are automatically closed even if errors occur.



Exception Handling for File I/O
Notes the importance of using 'try' and 'except' blocks to manage potential runtime errors during file operations, as seen in examples on exception handling.



Fetching Something From The Web
Extends file concepts to include reading data directly from a web URL, treating a web resource like a file.

FILES IN OTHER LEARNING RESOURCES



Dive Into Python 3
Chapter 13: 'Files'

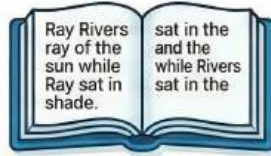


Principled Programming
Focuses on language-agnostic programming principles, no dedicated file handling chapter.

Word Frequencies

How to Count Word Frequencies in Python: A Step-by-Step Guide

The Objective: From Text to a Frequency Table



Input Text Example



What is a Word Frequency Count?
A data structure pairing each unique word with the number of times it appears.

Word Frequency Count

Word	Frequency Count
Ray	2
Rivers	2
sat	2
in	2
the	3
ray	1
of	1
sun	1
while	1
shade	1

```
{ 'Ray': 2,  
  'the': 3,  
  'sat': 2,  
  ... }
```

The Desired Output

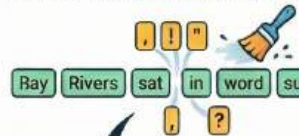
The Process: Building the Word Counter

Step 1: Define a Function and Read the File



Reads entire content into a string.

Step 2: Split Text and Remove Punctuation



`'split()'` breaks into words.
`'word.strip(string.punctuation)'` removes punctuation.

Step 3: Count Frequencies with a Dictionary



The Counting Logic
If NOT in dictionary, add as new key with value 1. If IS in dictionary, increment value by 1.

Step 4: Return the Result



Return the completed frequency dictionary.

Optional Enhancement: Sorting the Results

Why Sort? Raw dictionary output can be long and unordered.



1. Convert Dictionary to a List of Tuples

```
(2, 'Ray'), (3, 'the'), (2, 'sat')
```

Transform data into list of `(frequency, word)` tuples.

2. Sort the List

```
(2, 'Ray'), (3, 'the'), (2, 'sat'),  
(2, 'the'), (3, 'the'), (2, 'sat'),  
(2, 'Ray'), (3, 'the'), (2, 'sat')
```

Use `'sort()'` in reverse order for most frequent words first.

3. Print the Sorted List

```
Ray: 2  
the: 3  
sat: 2  
...
```

Loop through sorted list and print each word and its count in neat columns.

CSV Data Analysis

CSV Data Analysis

- Key Concepts in Data Representation

- + Discusses methods of representing tabular data in Python
- + Traditional approaches involve complex data structures that may not preserve data integrity

- Recommended Data Structure

- + Using a **list-of-dictionaries** as a more robust method for storing CSV data
- + Each dictionary represents a row, with column headers serving as dictionary keys

- CSV Reading Functionality

- + Provides a Python function 'read_csv()' designed to process CSV files
- + Function features:
 - Opens CSV file
 - Reads column headers
 - Converts rows into dictionary format
 - Returns field names and table data

- Implementation Details

- + Uses 'csv.DictReader' for efficient data parsing
- + Creates an **empty table** and **appends** row dictionaries
- + Closes file after processing

- Purpose

- + Aims to simplify CSV data handling
- + Provides a clean, structured approach to data representation

- Key Techniques for CSV Handling:

- Uses 'csv.DictReader' for reading CSV files
- Converts CSV data into a list of dictionaries
- Each dictionary represents a row with column headers as keys

Data Representation Strategy

- + The code demonstrates a more robust method of representing tabular data
 - Avoids potential data corruption when swapping order
 - Provides a flexible structure for data manipulation

Function Structure

- + The code includes a function for reading CSV files
- + Parameters:
 - 'csv_filename': Name of the CSV file to read
- + Returns:
 - List of field names (column headers)
 - List of dictionaries containing row data

Classes

Understanding Classes & Objects: A Programmer's Study Guide

THE BASICS: DEFINING CLASSES & CREATING OBJECTS



The User-Defined Class: A Blueprint for Objects
A class is a template created by a programmer for creating objects, providing initial values for attributes (data) and implementations of behavior (methods).



What is Object-Oriented Programming (OOP)?
A programming paradigm based on the concept of "objects", which can contain data (attributes) and code (methods).

Adding Methods: Defining Object Behavior
Methods are functions that are defined inside a class and describe the actions an object can perform.

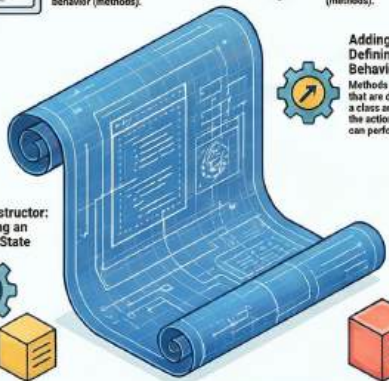


The Constructor: Initializing an Object's State



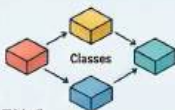
The Constructor

A special method that is called when a new object is created, used to set up its initial attributes.



Creating Instances: From Blueprint to Object

An "instance" is a specific object created from a particular class. The process is known as instantiation.

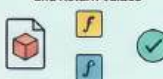


The Standard Approach: Classes as a Core Topic
Most texts introduce classes after fundamentals like data types, functions, and control flow are established, treating them as a central part of modern programming.

(Source: "Give Intro Python 3", "How to Think...")

WORKING WITH OBJECTS

Objects as Arguments and Return Values



Entire objects can be passed into functions as parameters and returned as results, just like any other data type.

Objects are Mutable



The state (attributes) of an object can typically be changed after it has been created.

Understanding "Sameness"



Equivalence
Differentiating between two variables that refer to the exact same object (identity) versus two different objects that happen to have the same attribute values (equivalence).



String Representation of an Object

You can define a special method to control how an object is represented as a string, which is useful for printing and debugging.



Defining Arithmetic Methods

Classes can define how standard operators like +, -, * work with their objects, a concept known as operator overloading.

TWO PHILOSOPHIES FOR LEARNING CLASSES

The "Principled" Approach: Classes as an Advanced Topic

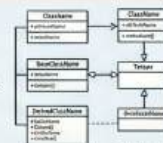
This methodology defers classes until the end, focusing first on universal principles like specifications, invariants, and refinement to build a strong foundation before introducing object-oriented constructs.

(Source: "Principled Programming")



Visualizing Relationships with Class Diagrams

Standardized diagrams used in software engineering to illustrate the structure of a system, including its classes, attributes, methods, and the relationships between objects.



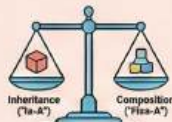
ADVANCED TOPICS: INHERITANCE & COMPOSITION

The Pillars of Object-Oriented Programming



Inheritance: The "Is-A" Relationship

A mechanism where a new class (subclass) derives attributes and methods from an existing class (superclass), allowing for code reuse and specialization.



Composition: The "Has-A" Relationship



A design principle where a class is constructed from other objects, building complex functionality by combining simpler parts.

Choosing Between Inheritance and Composition

A crucial design decision that involves weighing the benefits of a hierarchical "is-a" relationship against a more flexible "has-a" relationship.

Tkinter

Key Concepts of GUI Programming

- Event-driven programming is fundamental to GUI design
- Users interact with software through various **interactive elements**
- Programs must be prepared to **respond to user actions** immediately

Core Terminology

- **Window**: The primary rectangular area for GUI
- **Widget**: A rectangular region within a window for organizing elements
- **Widget**: General term for interactive GUI components like buttons, labels, text boxes
- **Layout**: Arrangement of widgets within window or frames
- **Callback**: Special function automatically triggered by user interaction

Programming Approach

- Tkinter is Python's standard module for creating graphical interfaces
- Object-oriented programming is recommended for single GUI applications
- Typical GUI program consists of:
 - **Import statements** + **Class definition**
 - **Initialization methods** + **Widget creation and placement**

Layout Techniques

- Grid method is a primary technique for positioning widgets
- Widgets placed in specific rows and columns; placement is resolved to fit uniformly

Tkinter Canvas Widget

Key Canvas Capabilities

- **Canvas Objects**: A versatile widget for displaying and manipulating various graphical shapes
- Primarily used for drawing and manipulating visual elements
- Rich command set for internal manipulation

Drawing Techniques

- Line Creation: Offers line drawing options available
- Can create strings, images, and canvas text
- Supports numerous line attributes like:
 - Color changes + provided color names
 - Width adjustments + smooth curve generation

Geometric Shape Options

- **Supported Shapes**
 - Lines with customizable properties
 - Rectangles with configurable attributes
 - Ovals with outline and fill options
 - Polygons for complex geometric designs

File and Folder Dialog Options

- **File Dialog Modes**: Supports file and directory interaction
- **askopen(filename)**: Select files to open
- **askopen(filename)**: Choose save location
- **askdirectory()**: Browse and select directory

EventHandler Class Characteristics

- Bases can be integrated into GUI program
- Typically attached to main Tk window
- Enables simple, user-friendly interaction mechanism

Practical Usage:

- Commonly used in callback function
- Triggered by user action like button click
- Help guide user interaction and provide feedback

Button widget Characteristics

- A Button is a clickable interface element that can display text or image
- Buttons are interactive components that typically trigger specific actions when clicked

Button Interaction Limitations

- Simply creating a button does not automatically define its behavior
- Developers must explicitly program the action to occur when a button is pressed

Additional Widget Attributes

- Widgets like Buttons and Frames can have configurable properties
- Background color can be set using hexadecimal or custom color string

Entry widget Functionality

- Entry widgets provide methods for text manipulation
- Includes operations like deleting and inserting text

Programming Considerations

- Widget placement (e.g., in grid layout) is essential for user interface clarity
- Each widget requires specific configuration to create interactive experience

Text and Textarea Widgets

- **Text Widgets**: Ability to add text directly onto canvas
- **Textarea Widgets**: Widgets can respond to mouse movements
- Color and width change while hovering

Practical Considerations

- Avoid using Canvas as a direct replacement for Frames
- Utilizing its extensive method set for dynamic graphical interfaces

Tkinter Orange Button and User Interaction

Key Dialog Box Types

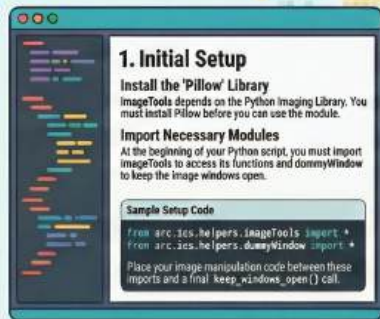
- **Message Boxes**: Provide informational, warning, or error notifications
 - Can display simple messages with titles and content
 - Examples include: **showinfo()**, **showerror()**, and **showwarning()**

Interactive Dialog Functionality

- **Question-Based Dialogs**: Allow user input and response selection
 - Support various interaction types like yes/no, ok/cancel, ok/yes/no
 - Return values based on user selection

Image Tools

A Developer's Guide to the Python imageTools Module



2. Loading and Displaying Images

Option A: Use the File Picker

Call the `pickAFile()` function, which opens a dialog window for you to select an image. It returns the file path as a string.

`File_path = pickAFile()`

Option B: Use a Direct File Path

For easier testing, you can pass a string containing the relative or absolute path to your image directly to the `Picture` constructor.

`pic = Picture(".../images/dummy.tif.jpg")`



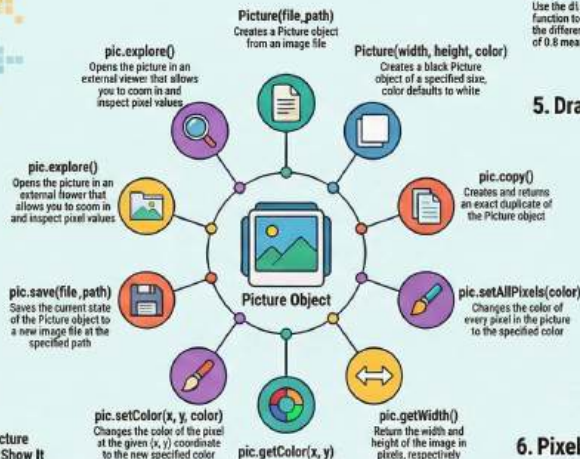
Picture

Create a Picture Object and Show It

Once you have a file path, create a `Picture` object. Then, call the `.show()` method on the object to display it in a new window.

`image = Picture(File_path)`
`image.show()`

3. Core Picture Methods



4. Working with Colors

Three Ways to Specify a Color

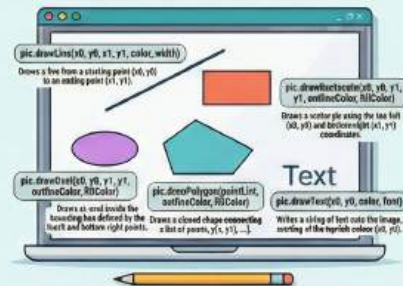
You can define colors using an RGB tuple, a color name string, or a hexadecimal string.

Color Tuple (RGB)	Color Name	Hex String
(255, 255, 255)	"white"	"#FFFFFF"
(0, 0, 0)	"black"	"#000000"
(255, 0, 0)	"red"	"#FF0000"
(0, 255, 0)	"lime"	"#00FF00"
(0, 0, 255)	"blue"	"#0000FF"
(255, 255, 0)	"yellow"	"#FFFF00"

Measuring Color Similarity

Use the `distance(color1, color2)` function to get a numeric value representing the difference between two colors. A value of 0.8 means the colors are identical.

5. Drawing Shapes and Text



6. Pixel-by-Pixel Manipulation

Loop Through Every Pixel

To access every pixel, you can iterate directly over the `Picture` object using a `for` loop. Each item in the loop will be a tuple containing an (x, y) coordinate.

Get and Set Pixel Colors

This loop iterates through each pixel, gets its color, increases the value of each RGB channel by 50, and sets the new color.

Code to Brighten an Image

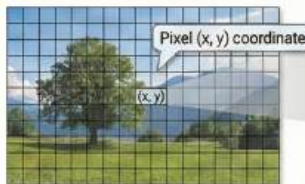
```
for (x, y) in picture:
    (r, g, b) = picture.getColor(x, y)
    new_color = (r + 50, g + 50, b + 50)
    picture.setColor(x, y, new_color)
```


Simple Images

A Python Guide to Simple Image Manipulation

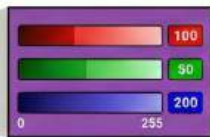
1. Setup & Core Concepts

A Digital Image is a Grid of Pixels



In this context, an image is a `Picture` object, which is a two-dimensional grid of pixels.

Each Pixel has an RGB Color Value



A pixel's color is defined by a tuple of three numbers (R, G, B). Each value ranges from 0 (no color) to 255 (full intensity).

Initial Setup for Image Processing

```
pip install Pillow
```

```
import imageTools, dummyWindow
```

You must install Pillow and import the custom `imageTools` and `dummyWindow` modules.

Loading and Displaying an Image



Color Specification Examples

Color Tuple	(255, 0, 0)	A tuple of three integers (R, G, B) from 0-255. This gives the most control.
Color Name	"red"	A string representing a common color name.
Hex String	"#FF0000"	A string starting with '#' followed by six hexadecimal digits representing RGB values.

2. The Pixel Manipulation Toolkit

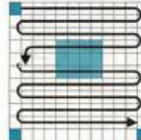
Always Work on a Copy



```
new_pic = pic.copy()
```

Use the `.copy()` method to create a duplicate. This preserves the original image.

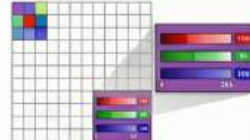
Loop Through Every Pixel



A `for` loop allows you to access the coordinates of every pixel in the image, one by one.

Code: `for (x, y) in new_pic:`

Set a Pixel's New Color



Set a Pixel's New Color

```
new_pic.setColor(x, y,
                 (new_r, new_g, new_b))
```

3. Common Image Effects

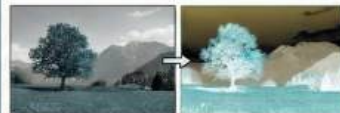


Grayscale (Simple Average)

```
lumin = (r + g + b) / 3
```

New Color: (lumin, lumin, lumin)

Code: (lumin, lumin, lumin)



Grayscale (Weighted Average)

```
lumin = w1*r + w2*g + w3*b
```

new_value = 255 - original_value

New Color: (255-r, 255-g, 255-b)

Shuffling Color Channels



Original (r, g, b) becomes

New Color: (b, r, g)

Separating Color Channels



Red channel Green channel Blue channel

Example: Set green and blue to zero

New Color: (r, 0, 0)

Altering a Single Channel



Example: Remove blue channel

New Color: (r, g, 0)

Multiple Images

Pixel Perfect: A Visual Guide to Image Manipulation in Python

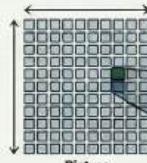
THE BUILDING BLOCKS: SETUP & BASICS



Initial Setup:
Import 'ImageTools'
& 'dummyWindow'.



Loading an Image:
File into 'Picture'
object using Me path
or 'pickAFile()'.



What is a Picture?
A grid of pixels.
'pic.getWidth()' &
'pic.getHeight()'.



What is a Pixel?
Color via RGB tuple (R, G, B).

COLOR TRANSFORMATIONS (Per-Pixel Effects)



Original

Grayscale
(Luminance Method)



Weighted Grayscale



Separating Color Channels



Isolate single channel
by setting others to 0.

$$\begin{matrix} r & g & b \\ \downarrow & \downarrow & \downarrow \\ \text{lumin} = \frac{r + g + b}{3} \end{matrix}$$

Accessing Every Pixel:
Loop through coordinates



Channel Shuffling



Color Manipulation

Color Tuple (RGB)	Color Name	Hex String
(255, 0, 0)	"red"	#FF0000
(0, 255, 0)	"green"	#00FF00
(0, 0, 255)	"blue"	#0000FF
(255, 255, 255)	"white"	#FFFFFF
(0, 0, 0)	"black"	#000000

GEOMETRIC TRANSFORMATIONS (Changing Pixel Positions)

Scaling Down



Source Image

New image half dimensions. Skip every other row and column from



Half Size

Scaling Up



Source Image

Double Size

New image twice dimensions. Copy each pixel to a 2x2 block.

Rotating Left (90°)



Source Image
(Width x Height)



(y, width - x - 1)

Rotated Left
(90°)

for (x, y) in picture:



$$y = -2x1$$

Channel Shuffling



R → G
G → B
B → R

Separating Color Channels



Isolate single channel by setting others to 0.

MULTI-IMAGE OPERATIONS



Copying an Image Onto Another

Loop through small image pixels. Copy to larger image at (start_x + x, start_y + y).



Cropping an Image

Create new black picture of desired size. Copy corresponding region.



Simple Blending
Superimposes two images of the same size. For each pixel position, the new color is the average of the colors from the two source images.



Weighted Blending
Blends two images, allowing one to dominate. A weight determines its prominence.

$$\text{avg_red} = (\text{wgt1} * \text{r1}) + ((1.0 - \text{wgt1}) * \text{r2})$$

Recursion

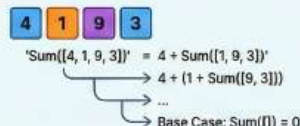
A Programmer's Guide to Recursion

What is Recursion?

A Method of Problem-Solving Where a Function Calls Itself

Recursion breaks a complex problem into smaller, more manageable sub-problems that are identical in nature to the original problem.

Recursion in Action: Core Examples



Calculating the Sum of a List

A classic example where the sum of a list is defined as the first element plus the sum of the rest of the list. The base case is an empty list, which has a sum of zero.

Converting an Integer to a String in Any Base



Recursion can be used to convert a number to binary, hexadecimal, or any other base by repeatedly dividing the number by the base and using the remainders to build the string.

The Recursive Leap of Faith

To solve a problem, a recursive function assumes it can already solve smaller versions of the same problem and uses that to solve the larger one.

The Three Laws of Recursion The Foundation of Every Recursive Algorithm

For a recursive function to work correctly and not run indefinitely, it must adhere to these three fundamental laws.

Law 1: A Recursive Algorithm Must Have a Base Case

The base case is a condition that stops the recursion. It's the simplest version of the problem that can be solved directly.

Law 2: A Recursive Algorithm Must Change Its State

The function must modify its input in a way that moves it closer to the base case with each recursive call.

THE ORIGINAL PROBLEM

RECURSIVE CALL 1 (Sub-problem)

RECURSIVE CALL 2 (Smaller Sub-problem)

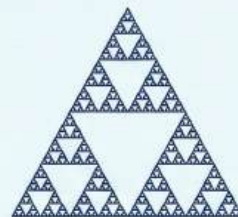
BASE CASE (Simplest Version)

Visualizing Recursion



Understanding the Recursive Call Stack

Visualizing how each function call is placed on a stack helps to trace the execution and understand how the function "remembers" where to return after each call.



Generating Fractals: The Sierpinski Triangle

This famous fractal provides a powerful visual demonstration of recursion. A large triangle is drawn by recursively drawing three smaller triangles within it.